```
import os
import cv2
import numpy as np
import tifffile as tiff
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F

from torch.utils.data import Dataset
from torch.utils.data import DataLoader
from torch.utils.data import random_split

from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay

from transformers import (
    SegformerForSemanticSegmentation
)

device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print("Device:", device)
```

```
Device: cuda
```

```
root_dir =
"/kaggle/input/datasets/rishikeshgopal121/landslide/ReadyToBeUsedDatas
et"

class SegFormerDataset(Dataset):

    def __init__(self, root_dir):

        self.samples = []

        for region in os.listdir(
            os.path.join(root_dir, "6days")
        ):

            region_path = os.path.join(
                root_dir,
                "6days",
                region
            )

            folders = os.listdir(region_path)
```

```
            for folder in folders:

                if folder == "Segmentations":
                    continue

                coh_path = os.path.join(
                    region_path,
                    folder,
                    "coherence"
                )

                phase_path = os.path.join(
                    region_path,
                    folder,
                    "phase"
                )

                mask_path = os.path.join(
                    region_path,
                    "Segmentations",
                    folder
                )

                if not (
                    os.path.exists(coh_path)
                    and
                    os.path.exists(phase_path)
                    and
                    os.path.exists(mask_path)
                ):
                    continue

                files = sorted(
                    os.listdir(coh_path)
                )

                for file in files:

                    if file.endswith(".tif"):

                        self.samples.append(

                            (

                                os.path.join(
                                    coh_path,
                                    file
                                ),
```

```
                                os.path.join(
                                    phase_path,
                                    file
                                ),

                                os.path.join(
                                    mask_path,
                                    file
                                )

                            )

                        )

        print(
            "Total Samples:",
            len(self.samples)
        )

    def __len__(self):

        return len(self.samples)

    def __getitem__(self, idx):

        coh_file, phase_file, mask_file = self.samples[idx]

        # ========================================
        # LOAD TIFF FILES
        # ========================================

        coherence = tiff.imread(
            coh_file
        ).astype(np.float32)

        phase = tiff.imread(
            phase_file
        ).astype(np.float32)

        mask = tiff.imread(
            mask_file
        ).astype(np.float32)

        # ========================================
        # CLAHE ON COHERENCE
        # ========================================

        coh_min = coherence.min()

        coh_max = coherence.max()
```

```
        coherence_norm = (
            coherence - coh_min
        ) / (
            coh_max - coh_min + 1e-8
        )
        coherence_uint8 = (
            coherence_norm * 255
        ).astype(np.uint8)
        clahe = cv2.createCLAHE(
            clipLimit=2.0,
            tileGridSize=(8,8)
        )
        coherence = clahe.apply(
            coherence_uint8
        ).astype(np.float32)
        coherence = coherence / 255.0

        # ========================================
        # PHASE DECOMPOSITION
        # ========================================

        cos_phase = np.cos(
            phase
        )
        sin_phase = np.sin(
            phase
        )

        # ========================================
        # 3-CHANNEL INPUT
        # ========================================

        image = np.stack(
            [
```

```
            coherence,
            cos_phase,
            sin_phase
        ],
        axis=0
    )
    mask = np.expand_dims(
        mask,
        axis=0
    )
    # ==========================================
    # HORIZONTAL FLIP
    # ==========================================
    if np.random.rand() > 0.5:
        image = np.flip(
            image,
            axis=2
        ).copy()
        mask = np.flip(
            mask,
            axis=2
        ).copy()
    # ==========================================
    # VERTICAL FLIP
    # ==========================================
    if np.random.rand() > 0.5:
        image = np.flip(
```

```
            image,
            axis=1
        ).copy()
        mask = np.flip(
            mask,
            axis=1
        ).copy()

    # ============================================
    # ROTATION
    # ============================================

    k = np.random.randint(
        0,
        4
    )

    image = np.rot90(
        image,
        k,
        axes=(1,2)
    ).copy()

    mask = np.rot90(
        mask,
        k,
        axes=(1,2)
    ).copy()

    # ============================================
    # GAUSSIAN NOISE
    # ============================================

    if np.random.rand() > 0.5:
        noise = np.random.normal(
```

```
                0,
                0.02,
                image[0].shape
            )
            image[0] += noise

        # ========================================
        # RANDOM BRIGHTNESS
        # ========================================
        if np.random.rand() > 0.5:
            factor = np.random.uniform(
                0.9,
                1.1
            )
            image[0] *= factor

        # ========================================
        # RANDOM CROP + RESIZE
        # ========================================
        if np.random.rand() > 0.5:
            crop_size = 80
            start_x = np.random.randint(
                0,
                100 - crop_size
            )
            start_y = np.random.randint(
                0,
                100 - crop_size
            )
            image_crop = image[
```

```
            :,
            start_y:start_y+crop_size,
            start_x:start_x+crop_size
        ]

        mask_crop = mask[
            :,
            start_y:start_y+crop_size,
            start_x:start_x+crop_size
        ]

        image_crop = torch.tensor(

            image_crop,

            dtype=torch.float32

        ).unsqueeze(0)

        mask_crop = torch.tensor(

            mask_crop,

            dtype=torch.float32

        ).unsqueeze(0)

        image_crop = F.interpolate(

            image_crop,

            size=(100,100),

            mode="bilinear",

            align_corners=False

        )

        mask_crop = F.interpolate(

            mask_crop,

            size=(100,100),

            mode="nearest"

        )

        image = image_crop.squeeze(0).numpy()
```

```
            mask = mask_crop.squeeze(0).numpy()

        # ========================================
        # Z-SCORE NORMALIZATION
        # ========================================

        image = image.astype(
            np.float32
        )

        for c in range(
            image.shape[0]
        ):

            mean = image[c].mean()

            std = image[c].std()

            image[c] = (

                image[c] - mean

            ) / (

                std + 1e-8

            )

        mask = mask.astype(
            np.float32
        )

        return (

            torch.tensor(
                image,
                dtype=torch.float32
            ),

            torch.tensor(
                mask,
                dtype=torch.float32
            )

        )

dataset = SegFormerDataset(
    root_dir
)

Total Samples: 4655
```

```
train_size = int(0.70 * len(dataset))

val_size = int(0.15 * len(dataset))

test_size = len(dataset) - train_size - val_size

train_dataset, val_dataset, test_dataset = random_split(

    dataset,

    [
        train_size,
        val_size,
        test_size
    ]
)

print("Train:", len(train_dataset))
print("Validation:", len(val_dataset))
print("Test:", len(test_dataset))
```

```
Train: 3258
Validation: 698
Test: 699
```

```
train_loader = DataLoader(

    train_dataset,

    batch_size=8,

    shuffle=True
)

test_loader = DataLoader(

    test_dataset,

    batch_size=8,

    shuffle=False
)

val_loader = DataLoader(

    val_dataset,

    batch_size=8,

    shuffle=False
)
```

```
class SegFormerModel(nn.Module):

    def __init__(self):

        super().__init__()

        self.segformer =
SegformerForSemanticSegmentation.from_pretrained(

            "nvidia/segformer-b0-finetuned-ade-512-512",

            num_labels=1,

            ignore_mismatched_sizes=True

        )

    def forward(self, x):

        outputs = self.segformer(

            pixel_values=x

        )

        logits = outputs.logits

        logits = F.interpolate(

            logits,

            size=(100, 100),

            mode="bilinear",

            align_corners=False

        )

        return logits

model = SegFormerModel().to(device)

print(model)
```

```
Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.
```


```
{"model_id":"3cb375fdbd7c45d9a6231f6036215d5e","version_major":2,"vers
ion_minor":0}
```

{"model_id":"6746ed69a0fb4709b20210d1b80c26a9","version_major":2,"version_minor":0}

{"model_id":"f82f133f2723453fbb4c85f70f371298","version_major":2,"version_minor":0}

```
SegformerForSemanticSegmentation LOAD REPORT from: nvidia/segformer-
b0-finetuned-ade-512-512
Key                             | Status   |

--------------------------------+----------
+-------------------------------------------------------------------------
-----------------------------
decode_head.classifier.weight | MISMATCH | Reinit due to size mismatch
ckpt: torch.Size([150, 256, 1, 1]) vs model:torch.Size([1, 256, 1, 1])
decode_head.classifier.bias   | MISMATCH | Reinit due to size mismatch
ckpt: torch.Size([150]) vs model:torch.Size([1])

Notes:
- MISMATCH :ckpt weights were loaded, but they did not match the
original empty weight shapes.

SegFormerModel(
  (segformer): SegformerForSemanticSegmentation(
    (segformer): SegformerModel(
      (encoder): SegformerEncoder(
        (patch_embeddings): ModuleList(
          (0): SegformerOverlapPatchEmbeddings(
            (proj): Conv2d(3, 32, kernel_size=(7, 7), stride=(4, 4),
padding=(3, 3))
            (layer_norm): LayerNorm((32,), eps=1e-05,
elementwise_affine=True)
          )
          (1): SegformerOverlapPatchEmbeddings(
            (proj): Conv2d(32, 64, kernel_size=(3, 3), stride=(2, 2),
padding=(1, 1))
            (layer_norm): LayerNorm((64,), eps=1e-05,
elementwise_affine=True)
          )
          (2): SegformerOverlapPatchEmbeddings(
            (proj): Conv2d(64, 160, kernel_size=(3, 3), stride=(2, 2),
padding=(1, 1))
            (layer_norm): LayerNorm((160,), eps=1e-05,
elementwise_affine=True)
          )
          (3): SegformerOverlapPatchEmbeddings(
            (proj): Conv2d(160, 256, kernel_size=(3, 3), stride=(2,
2), padding=(1, 1))
            (layer_norm): LayerNorm((256,), eps=1e-05,
elementwise_affine=True)
```

```
          )
        )
        (block): ModuleList(
          (0): ModuleList(
            (0): SegformerLayer(
              (layer_norm_1): LayerNorm((32,), eps=1e-05,
elementwise_affine=True)
              (attention): SegformerAttention(
                (self): SegformerEfficientSelfAttention(
                  (query): Linear(in_features=32, out_features=32,
bias=True)
                  (key): Linear(in_features=32, out_features=32,
bias=True)
                  (value): Linear(in_features=32, out_features=32,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                  (sr): Conv2d(32, 32, kernel_size=(8, 8), stride=(8,
8))
                  (layer_norm): LayerNorm((32,), eps=1e-05,
elementwise_affine=True)
                )
                (output): SegformerSelfOutput(
                  (dense): Linear(in_features=32, out_features=32,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                )
              )
              (drop_path): Identity()
              (layer_norm_2): LayerNorm((32,), eps=1e-05,
elementwise_affine=True)
              (mlp): SegformerMixFFN(
                (dense1): Linear(in_features=32, out_features=128,
bias=True)
                (dwconv): SegformerDWConv(
                  (dwconv): Conv2d(128, 128, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=128)
                )
                (intermediate_act_fn): GELUActivation()
                (dense2): Linear(in_features=128, out_features=32,
bias=True)
                (dropout): Dropout(p=0.0, inplace=False)
              )
            )
            (1): SegformerLayer(
              (layer_norm_1): LayerNorm((32,), eps=1e-05,
elementwise_affine=True)
              (attention): SegformerAttention(
                (self): SegformerEfficientSelfAttention(
                  (query): Linear(in_features=32, out_features=32,
```

```
bias=True)
                  (key): Linear(in_features=32, out_features=32,
bias=True)
                  (value): Linear(in_features=32, out_features=32,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                  (sr): Conv2d(32, 32, kernel_size=(8, 8), stride=(8,
8))
                  (layer_norm): LayerNorm((32,), eps=1e-05,
elementwise_affine=True)
                )
                (output): SegformerSelfOutput(
                  (dense): Linear(in_features=32, out_features=32,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                )
              )
              (drop_path): SegformerDropPath(p=0.014285714365541935)
              (layer_norm_2): LayerNorm((32,), eps=1e-05,
elementwise_affine=True)
              (mlp): SegformerMixFFN(
                (dense1): Linear(in_features=32, out_features=128,
bias=True)
                (dwconv): SegformerDWConv(
                  (dwconv): Conv2d(128, 128, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=128)
                )
                (intermediate_act_fn): GELUActivation()
                (dense2): Linear(in_features=128, out_features=32,
bias=True)
                (dropout): Dropout(p=0.0, inplace=False)
              )
            )
          )
          (1): ModuleList(
            (0): SegformerLayer(
              (layer_norm_1): LayerNorm((64,), eps=1e-05,
elementwise_affine=True)
              (attention): SegformerAttention(
                (self): SegformerEfficientSelfAttention(
                  (query): Linear(in_features=64, out_features=64,
bias=True)
                  (key): Linear(in_features=64, out_features=64,
bias=True)
                  (value): Linear(in_features=64, out_features=64,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                  (sr): Conv2d(64, 64, kernel_size=(4, 4), stride=(4,
4))
```

```
                  (layer_norm): LayerNorm((64,), eps=1e-05,
elementwise_affine=True)
                )
                (output): SegformerSelfOutput(
                  (dense): Linear(in_features=64, out_features=64,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                )
              )
              (drop_path): SegformerDropPath(p=0.02857142873108387)
              (layer_norm_2): LayerNorm((64,), eps=1e-05,
elementwise_affine=True)
              (mlp): SegformerMixFFN(
                (dense1): Linear(in_features=64, out_features=256,
bias=True)
                (dwconv): SegformerDWConv(
                  (dwconv): Conv2d(256, 256, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=256)
                )
                (intermediate_act_fn): GELUActivation()
                (dense2): Linear(in_features=256, out_features=64,
bias=True)
                (dropout): Dropout(p=0.0, inplace=False)
              )
            )
            (1): SegformerLayer(
              (layer_norm_1): LayerNorm((64,), eps=1e-05,
elementwise_affine=True)
              (attention): SegformerAttention(
                (self): SegformerEfficientSelfAttention(
                  (query): Linear(in_features=64, out_features=64,
bias=True)
                  (key): Linear(in_features=64, out_features=64,
bias=True)
                  (value): Linear(in_features=64, out_features=64,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                  (sr): Conv2d(64, 64, kernel_size=(4, 4), stride=(4,
4))
                  (layer_norm): LayerNorm((64,), eps=1e-05,
elementwise_affine=True)
                )
                (output): SegformerSelfOutput(
                  (dense): Linear(in_features=64, out_features=64,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                )
              )
              (drop_path): SegformerDropPath(p=0.04285714402794838)
```

```
              (layer_norm_2): LayerNorm((64,), eps=1e-05,
elementwise_affine=True)
              (mlp): SegformerMixFFN(
                (dense1): Linear(in_features=64, out_features=256,
bias=True)
                (dwconv): SegformerDWConv(
                  (dwconv): Conv2d(256, 256, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=256)
                )
                (intermediate_act_fn): GELUActivation()
                (dense2): Linear(in_features=256, out_features=64,
bias=True)
                (dropout): Dropout(p=0.0, inplace=False)
              )
            )
          )
          (2): ModuleList(
            (0): SegformerLayer(
              (layer_norm_1): LayerNorm((160,), eps=1e-05,
elementwise_affine=True)
              (attention): SegformerAttention(
                (self): SegformerEfficientSelfAttention(
                  (query): Linear(in_features=160, out_features=160,
bias=True)
                  (key): Linear(in_features=160, out_features=160,
bias=True)
                  (value): Linear(in_features=160, out_features=160,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                  (sr): Conv2d(160, 160, kernel_size=(2, 2),
stride=(2, 2))
                  (layer_norm): LayerNorm((160,), eps=1e-05,
elementwise_affine=True)
                )
                (output): SegformerSelfOutput(
                  (dense): Linear(in_features=160, out_features=160,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                )
              )
              (drop_path): SegformerDropPath(p=0.05714285746216774)
              (layer_norm_2): LayerNorm((160,), eps=1e-05,
elementwise_affine=True)
              (mlp): SegformerMixFFN(
                (dense1): Linear(in_features=160, out_features=640,
bias=True)
                (dwconv): SegformerDWConv(
                  (dwconv): Conv2d(640, 640, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=640)
```

```
                )
                (intermediate_act_fn): GELUActivation()
                (dense2): Linear(in_features=640, out_features=160,
bias=True)
                (dropout): Dropout(p=0.0, inplace=False)
              )
            )
            (1): SegformerLayer(
              (layer_norm_1): LayerNorm((160,), eps=1e-05,
elementwise_affine=True)
              (attention): SegformerAttention(
                (self): SegformerEfficientSelfAttention(
                  (query): Linear(in_features=160, out_features=160,
bias=True)
                  (key): Linear(in_features=160, out_features=160,
bias=True)
                  (value): Linear(in_features=160, out_features=160,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                  (sr): Conv2d(160, 160, kernel_size=(2, 2),
stride=(2, 2))
                  (layer_norm): LayerNorm((160,), eps=1e-05,
elementwise_affine=True)
                )
                (output): SegformerSelfOutput(
                  (dense): Linear(in_features=160, out_features=160,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                )
              )
              (drop_path): SegformerDropPath(p=0.0714285746216774)
              (layer_norm_2): LayerNorm((160,), eps=1e-05,
elementwise_affine=True)
              (mlp): SegformerMixFFN(
                (dense1): Linear(in_features=160, out_features=640,
bias=True)
                (dwconv): SegformerDWConv(
                  (dwconv): Conv2d(640, 640, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=640)
                )
                (intermediate_act_fn): GELUActivation()
                (dense2): Linear(in_features=640, out_features=160,
bias=True)
                (dropout): Dropout(p=0.0, inplace=False)
              )
            )
          )
          (3): ModuleList(
            (0): SegformerLayer(
```

```
              (layer_norm_1): LayerNorm((256,), eps=1e-05,
elementwise_affine=True)
              (attention): SegformerAttention(
                (self): SegformerEfficientSelfAttention(
                  (query): Linear(in_features=256, out_features=256,
bias=True)
                  (key): Linear(in_features=256, out_features=256,
bias=True)
                  (value): Linear(in_features=256, out_features=256,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                )
                (output): SegformerSelfOutput(
                  (dense): Linear(in_features=256, out_features=256,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                )
              )
              (drop_path): SegformerDropPath(p=0.08571428805589676)
              (layer_norm_2): LayerNorm((256,), eps=1e-05,
elementwise_affine=True)
              (mlp): SegformerMixFFN(
                (dense1): Linear(in_features=256, out_features=1024,
bias=True)
                (dwconv): SegformerDWConv(
                  (dwconv): Conv2d(1024, 1024, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=1024)
                )
                (intermediate_act_fn): GELUActivation()
                (dense2): Linear(in_features=1024, out_features=256,
bias=True)
                (dropout): Dropout(p=0.0, inplace=False)
              )
            )
            (1): SegformerLayer(
              (layer_norm_1): LayerNorm((256,), eps=1e-05,
elementwise_affine=True)
              (attention): SegformerAttention(
                (self): SegformerEfficientSelfAttention(
                  (query): Linear(in_features=256, out_features=256,
bias=True)
                  (key): Linear(in_features=256, out_features=256,
bias=True)
                  (value): Linear(in_features=256, out_features=256,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                )
                (output): SegformerSelfOutput(
                  (dense): Linear(in_features=256, out_features=256,
```

```
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                )
              )
              (drop_path): SegformerDropPath(p=0.10000000149011612)
              (layer_norm_2): LayerNorm((256,), eps=1e-05,
elementwise_affine=True)
              (mlp): SegformerMixFFN(
                (dense1): Linear(in_features=256, out_features=1024,
bias=True)
                (dwconv): SegformerDWConv(
                  (dwconv): Conv2d(1024, 1024, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=1024)
                )
                (intermediate_act_fn): GELUActivation()
                (dense2): Linear(in_features=1024, out_features=256,
bias=True)
                (dropout): Dropout(p=0.0, inplace=False)
              )
            )
          )
        )
        (layer_norm): ModuleList(
          (0): LayerNorm((32,), eps=1e-05, elementwise_affine=True)
          (1): LayerNorm((64,), eps=1e-05, elementwise_affine=True)
          (2): LayerNorm((160,), eps=1e-05, elementwise_affine=True)
          (3): LayerNorm((256,), eps=1e-05, elementwise_affine=True)
        )
      )
    )
    (decode_head): SegformerDecodeHead(
      (linear_c): ModuleList(
        (0): SegformerMLP(
          (proj): Linear(in_features=32, out_features=256, bias=True)
        )
        (1): SegformerMLP(
          (proj): Linear(in_features=64, out_features=256, bias=True)
        )
        (2): SegformerMLP(
          (proj): Linear(in_features=160, out_features=256, bias=True)
        )
        (3): SegformerMLP(
          (proj): Linear(in_features=256, out_features=256, bias=True)
        )
      )
      (linear_fuse): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1,
1), bias=False)
      (batch_norm): BatchNorm2d(256, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True)
      (activation): ReLU()
```

```
      (dropout): Dropout(p=0.1, inplace=False)
      (classifier): Conv2d(256, 1, kernel_size=(1, 1), stride=(1, 1))
    )
  )
)
```

```
sample_batch = next(iter(train_loader))

images, masks = sample_batch

images = images.to(device)

with torch.no_grad():

    outputs = model(images)

print("Input Shape :", images.shape)

print("Output Shape:", outputs.shape)
```

```
Input Shape : torch.Size([8, 3, 100, 100])
Output Shape: torch.Size([8, 1, 100, 100])
```

```
class DiceLoss(nn.Module):

    def __init__(self):

        super().__init__()

    def forward(self, pred, target):

        pred = torch.sigmoid(pred)

        intersection = (
            pred * target
        ).sum()

        dice = (

            2 * intersection

        ) / (

            pred.sum()
            +
            target.sum()
            +
            1e-8

        )
```

```
        return 1 - dice


class FocalLoss(nn.Module):

    def __init__(
        self,
        alpha=0.8,
        gamma=2
    ):

        super().__init__()

        self.alpha = alpha

        self.gamma = gamma

    def forward(
        self,
        pred,
        target
    ):

        bce = F.binary_cross_entropy_with_logits(

            pred,

            target,

            reduction='none'

        )

        pt = torch.exp(-bce)

        focal = (

            self.alpha

            *

            (1 - pt) ** self.gamma

            *

            bce

        )

        return focal.mean()
```

```
dice_loss = DiceLoss()

focal_loss = FocalLoss()


def hybrid_loss(
    pred,
    target
):

    d = dice_loss(
        pred,
        target
    )

    f = focal_loss(
        pred,
        target
    )

    return d + f

class DiceLoss(nn.Module):

    def __init__(self):

        super().__init__()

    def forward(self, pred, target):

        pred = torch.sigmoid(pred)

        intersection = (
            pred * target
        ).sum()

        dice = (

            2 * intersection

        ) / (

            pred.sum()
            +
            target.sum()
            +
            1e-8

        )
```

```
        return 1 - dice


class FocalLoss(nn.Module):

    def __init__(
        self,
        alpha=0.8,
        gamma=2
    ):

        super().__init__()

        self.alpha = alpha

        self.gamma = gamma

    def forward(
        self,
        pred,
        target
    ):

        bce = F.binary_cross_entropy_with_logits(

            pred,

            target,

            reduction='none'

        )

        pt = torch.exp(-bce)

        focal = (

            self.alpha

            *

            (1 - pt) ** self.gamma

            *

            bce

        )

        return focal.mean()
```

```
dice_loss = DiceLoss()

focal_loss = FocalLoss()


def hybrid_loss(
    pred,
    target
):

    d = dice_loss(
        pred,
        target
    )

    f = focal_loss(
        pred,
        target
    )

    return d + f

optimizer = optim.AdamW(

    model.parameters(),

    lr=0.0001,

    weight_decay=1e-4

)

scheduler = optim.lr_scheduler.ReduceLROnPlateau(

    optimizer,

    mode='min',

    factor=0.5,

    patience=5

)

train_losses = []

train_acc = []

train_prec = []

train_rec = []
```

```
epochs = 120
best_loss = float('inf')
for epoch in range(epochs):
    model.train()
    total_loss = 0
    correct = 0
    total = 0
    TP = 0
    FP = 0
    FN = 0
    print(f"\nEpoch {epoch+1}/{epochs}")
    for i, (images, masks) in enumerate(train_loader):
        images = images.to(device)
        masks = masks.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = hybrid_loss(
            outputs,
            masks
        )
        loss.backward()
        torch.nn.utils.clip_grad_norm_(
            model.parameters(),
            1.0
        )
        optimizer.step()
```

```
        total_loss += loss.item()
        # ================ METRICS ================
        pred = torch.sigmoid(outputs)
        pred_bin = (
            pred > 0.5
        ).float()
        correct += (
            pred_bin == masks
        ).sum().item()
        total += masks.numel()
        TP += (
            pred_bin * masks
        ).sum().item()
        FP += (
            pred_bin * (1 - masks)
        ).sum().item()
        FN += (
            (1 - pred_bin) * masks
        ).sum().item()
        progress = (
            (i+1)
            /
            len(train_loader)
        ) * 100
        print(
            f"\rProgress: {progress:.2f}% | Loss: {loss.item():.4f}",
```

```
            end=""
        )
    # ================ EPOCH METRICS ================
    acc = correct / total
    precision = TP / (
        TP + FP + 1e-8
    )
    recall = TP / (
        TP + FN + 1e-8
    )
    train_losses.append(
        total_loss
    )
    train_acc.append(
        acc
    )
    train_prec.append(
        precision
    )
    train_rec.append(
        recall
    )
    print(
        f"\nLoss: {total_loss:.4f}"
    )
    print(
```

```
        f"Accuracy : {acc:.4f}"
    )
    print(
        f"Precision: {precision:.4f}"
    )
    print(
        f"Recall   : {recall:.4f}"
    )
    scheduler.step(
        total_loss
    )
    if total_loss < best_loss:
        best_loss = total_loss
        torch.save(
            model.state_dict(),
            "segformer_best.pth"
        )
        print(
            "Model Saved!"
        )
```

```
Epoch 1/120
Progress: 100.00% | Loss: 0.9979
Loss: 349.4842
Accuracy : 0.9214
Precision: 0.2740
Recall   : 0.5985
Model Saved!

Epoch 2/120
Progress: 100.00% | Loss: 0.4329
```

```
Loss: 256.2527
Accuracy : 0.9666
Precision: 0.5776
Recall   : 0.5926
Model Saved!

Epoch 3/120
Progress: 100.00% | Loss: 1.0001
Loss: 216.0025
Accuracy : 0.9713
Precision: 0.6408
Recall   : 0.6137
Model Saved!

Epoch 4/120
Progress: 100.00% | Loss: 0.6971
Loss: 198.2104
Accuracy : 0.9732
Precision: 0.6721
Recall   : 0.6213
Model Saved!

Epoch 5/120
Progress: 100.00% | Loss: 0.5724
Loss: 190.7106
Accuracy : 0.9743
Precision: 0.6779
Recall   : 0.6452
Model Saved!

Epoch 6/120
Progress: 100.00% | Loss: 0.2892
Loss: 184.2680
Accuracy : 0.9745
Precision: 0.6820
Recall   : 0.6491
Model Saved!

Epoch 7/120
Progress: 100.00% | Loss: 0.3063
Loss: 177.3399
Accuracy : 0.9759
Precision: 0.7046
Recall   : 0.6669
Model Saved!

Epoch 8/120
Progress: 100.00% | Loss: 0.3310
Loss: 177.8449
Accuracy : 0.9761
```

```
Precision: 0.7050
Recall   : 0.6626

Epoch 9/120
Progress: 100.00% | Loss: 0.2560
Loss: 169.1503
Accuracy : 0.9769
Precision: 0.7165
Recall   : 0.6782
Model Saved!

Epoch 10/120
Progress: 100.00% | Loss: 0.2133
Loss: 168.9567
Accuracy : 0.9768
Precision: 0.7140
Recall   : 0.6808
Model Saved!

Epoch 11/120
Progress: 100.00% | Loss: 0.4704
Loss: 165.4182
Accuracy : 0.9775
Precision: 0.7237
Recall   : 0.6848
Model Saved!

Epoch 12/120
Progress: 100.00% | Loss: 0.4033
Loss: 161.4624
Accuracy : 0.9779
Precision: 0.7305
Recall   : 0.6947
Model Saved!

Epoch 13/120
Progress: 100.00% | Loss: 0.1971
Loss: 157.7553
Accuracy : 0.9784
Precision: 0.7328
Recall   : 0.7012
Model Saved!

Epoch 14/120
Progress: 100.00% | Loss: 0.1748
Loss: 156.4499
Accuracy : 0.9784
Precision: 0.7348
Recall   : 0.7037
Model Saved!
```

```
Epoch 15/120
Progress: 100.00% | Loss: 0.6473
Loss: 159.9100
Accuracy : 0.9785
Precision: 0.7367
Recall   : 0.6987

Epoch 16/120
Progress: 100.00% | Loss: 0.1578
Loss: 156.3230
Accuracy : 0.9792
Precision: 0.7456
Recall   : 0.7126
Model Saved!

Epoch 17/120
Progress: 100.00% | Loss: 0.3879
Loss: 152.9350
Accuracy : 0.9791
Precision: 0.7442
Recall   : 0.7171
Model Saved!

Epoch 18/120
Progress: 100.00% | Loss: 0.2285
Loss: 149.2312
Accuracy : 0.9797
Precision: 0.7493
Recall   : 0.7207
Model Saved!

Epoch 19/120
Progress: 100.00% | Loss: 0.6815
Loss: 147.1722
Accuracy : 0.9803
Precision: 0.7574
Recall   : 0.7296
Model Saved!

Epoch 20/120
Progress: 100.00% | Loss: 0.2939
Loss: 140.4947
Accuracy : 0.9811
Precision: 0.7738
Recall   : 0.7359

Epoch 25/120
Progress: 100.00% | Loss: 0.4162
Loss: 139.4635
```

```
Accuracy : 0.9814
Precision: 0.7769
Recall   : 0.7439
Model Saved!

Epoch 26/120
Progress: 100.00% | Loss: 0.0954
Loss: 138.1922
Accuracy : 0.9812
Precision: 0.7661
Recall   : 0.7484
Model Saved!

Epoch 27/120
Progress: 100.00% | Loss: 0.4080
Loss: 136.9629
Accuracy : 0.9816
Precision: 0.7754
Recall   : 0.7436
Model Saved!

Epoch 28/120
Progress: 100.00% | Loss: 0.6960
Loss: 133.6586
Accuracy : 0.9817
Precision: 0.7800
Recall   : 0.7471
Model Saved!

Epoch 29/120
Progress: 100.00% | Loss: 0.3770
Loss: 134.3125
Accuracy : 0.9820
Precision: 0.7775
Recall   : 0.7516

Epoch 30/120
Progress: 100.00% | Loss: 0.1271
Loss: 131.6302
Accuracy : 0.9820
Precision: 0.7787
Recall   : 0.7572
Model Saved!

Epoch 31/120
Progress: 100.00% | Loss: 0.4053
Loss: 131.9667
Accuracy : 0.9824
Precision: 0.7864
Recall   : 0.7553
```

```
Epoch 32/120
Progress: 100.00% | Loss: 0.2657
Loss: 131.1513
Accuracy : 0.9825
Precision: 0.7788
Recall   : 0.7604
Model Saved!

Epoch 33/120
Progress: 100.00% | Loss: 0.1557
Loss: 126.6789
Accuracy : 0.9827
Precision: 0.7848
Recall   : 0.7674
Model Saved!

Epoch 34/120
Progress: 100.00% | Loss: 0.2003
Loss: 129.9445
Accuracy : 0.9827
Precision: 0.7876
Recall   : 0.7656

Epoch 35/120
Progress: 100.00% | Loss: 0.3022
Loss: 127.9797
Accuracy : 0.9829
Precision: 0.7907
Recall   : 0.7685

Epoch 36/120
Progress: 100.00% | Loss: 0.2688
Loss: 125.8767
Accuracy : 0.9830
Precision: 0.7872
Recall   : 0.7720
Model Saved!

Epoch 37/120
Progress: 100.00% | Loss: 1.3064
Loss: 126.9732
Accuracy : 0.9835
Precision: 0.7946
Recall   : 0.7725

Epoch 38/120
Progress: 100.00% | Loss: 0.2030
Loss: 121.4888
Accuracy : 0.9835
```

```
Precision: 0.7959
Recall   : 0.7777
Model Saved!

Epoch 39/120
Progress: 100.00% | Loss: 0.1013
Loss: 125.8481
Accuracy : 0.9833
Precision: 0.7964
Recall   : 0.7676

Epoch 40/120
Progress: 100.00% | Loss: 1.0000
Loss: 123.0953
Accuracy : 0.9832
Precision: 0.7892
Recall   : 0.7811

Epoch 41/120
Progress: 100.00% | Loss: 0.2237
Loss: 122.0157
Accuracy : 0.9836
Precision: 0.7979
Recall   : 0.7782

Epoch 42/120
Progress: 100.00% | Loss: 0.2803
Loss: 118.8820
Accuracy : 0.9839
Precision: 0.8029
Recall   : 0.7782
Model Saved!

Epoch 43/120
Progress: 100.00% | Loss: 0.3856
Loss: 119.0518
Accuracy : 0.9839
Precision: 0.8059
Recall   : 0.7813

Epoch 44/120
Progress: 100.00% | Loss: 0.3294
Loss: 121.0868
Accuracy : 0.9837
Precision: 0.7990
Recall   : 0.7787

Epoch 45/120
Progress: 100.00% | Loss: 0.7096
Loss: 122.2009
```

```
Accuracy : 0.9838
Precision: 0.8004
Recall   : 0.7784

Epoch 46/120
Progress: 100.00% | Loss: 0.1373
Loss: 118.9858
Accuracy : 0.9842
Precision: 0.8087
Recall   : 0.7793

Epoch 47/120
Progress: 100.00% | Loss: 1.0058
Loss: 114.1066
Accuracy : 0.9849
Precision: 0.8124
Recall   : 0.7922
Model Saved!

Epoch 48/120
Progress: 100.00% | Loss: 0.6276
Loss: 110.9439
Accuracy : 0.9846
Precision: 0.8162
Recall   : 0.7902
Model Saved!

Epoch 49/120
Progress: 100.00% | Loss: 0.3809
Loss: 115.6144
Accuracy : 0.9847
Precision: 0.8099
Recall   : 0.7925

Epoch 50/120
Progress: 100.00% | Loss: 1.0252
Loss: 115.4985
Accuracy : 0.9846
Precision: 0.8091
Recall   : 0.7874

Epoch 51/120
Progress: 100.00% | Loss: 0.2854
Loss: 113.9267
Accuracy : 0.9846
Precision: 0.8111
Recall   : 0.7936

Epoch 52/120
Progress: 100.00% | Loss: 0.5383
```

```
Loss: 109.6864
Accuracy : 0.9852
Precision: 0.8183
Recall   : 0.7999
Model Saved!

Epoch 53/120
Progress: 100.00% | Loss: 0.2276
Loss: 110.7122
Accuracy : 0.9850
Precision: 0.8152
Recall   : 0.7971

Epoch 54/120
Progress: 100.00% | Loss: 0.2166
Loss: 108.2762
Accuracy : 0.9853
Precision: 0.8183
Recall   : 0.8027
Model Saved!

Epoch 55/120
Progress: 100.00% | Loss: 1.0000
Loss: 111.1041
Accuracy : 0.9849
Precision: 0.8160
Recall   : 0.7967

Epoch 56/120
Progress: 100.00% | Loss: 0.1247
Loss: 106.8916
Accuracy : 0.9855
Precision: 0.8234
Recall   : 0.7999
Model Saved!

Epoch 57/120
Progress: 100.00% | Loss: 1.0127
Loss: 110.3512
Accuracy : 0.9853
Precision: 0.8181
Recall   : 0.8010

Epoch 58/120
Progress: 100.00% | Loss: 0.1476
Loss: 108.3372
Accuracy : 0.9857
Precision: 0.8231
Recall   : 0.8060
```

```
Epoch 59/120
Progress: 100.00% | Loss: 1.0510
Loss: 107.5846
Accuracy : 0.9854
Precision: 0.8183
Recall   : 0.8080

Epoch 60/120
Progress: 100.00% | Loss: 0.5910
Loss: 111.6945
Accuracy : 0.9854
Precision: 0.8191
Recall   : 0.8001

Epoch 61/120
Progress: 100.00% | Loss: 0.1282
Loss: 105.5063
Accuracy : 0.9858
Precision: 0.8244
Recall   : 0.8090
Model Saved!

Epoch 62/120
Progress: 100.00% | Loss: 0.1905
Loss: 103.7617
Accuracy : 0.9860
Precision: 0.8267
Recall   : 0.8120
Model Saved!

Epoch 63/120
Progress: 100.00% | Loss: 0.5446
Loss: 107.8110
Accuracy : 0.9857
Precision: 0.8274
Recall   : 0.8039

Epoch 64/120
Progress: 100.00% | Loss: 0.4990
Loss: 105.0474
Accuracy : 0.9861
Precision: 0.8294
Recall   : 0.8100

Epoch 65/120
Progress: 100.00% | Loss: 0.1405
Loss: 102.2938
Accuracy : 0.9864
Precision: 0.8304
Recall   : 0.8154
```

```
Model Saved!

Epoch 66/120
Progress: 100.00% | Loss: 0.1799
Loss: 105.9473
Accuracy : 0.9860
Precision: 0.8297
Recall   : 0.8125

Epoch 67/120
Progress: 100.00% | Loss: 1.0000
Loss: 101.3751
Accuracy : 0.9864
Precision: 0.8292
Recall   : 0.8162
Model Saved!

Epoch 68/120
Progress: 100.00% | Loss: 0.8555
Loss: 100.9445
Accuracy : 0.9864
Precision: 0.8348
Recall   : 0.8170
Model Saved!

Epoch 69/120
Progress: 100.00% | Loss: 1.0328
Loss: 102.2456
Accuracy : 0.9865
Precision: 0.8308
Recall   : 0.8216

Epoch 70/120
Progress: 100.00% | Loss: 1.0208
Loss: 102.0775
Accuracy : 0.9863
Precision: 0.8311
Recall   : 0.8140

Epoch 71/120
Progress: 100.00% | Loss: 0.2472
Loss: 100.2548
Accuracy : 0.9864
Precision: 0.8383
Recall   : 0.8120
Model Saved!

Epoch 72/120
Progress: 100.00% | Loss: 0.3008
Loss: 97.4431
```

```
Accuracy : 0.9867
Precision: 0.8350
Recall   : 0.8200
Model Saved!

Epoch 73/120
Progress: 100.00% | Loss: 0.2746
Loss: 102.7468
Accuracy : 0.9862
Precision: 0.8306
Recall   : 0.8169

Epoch 74/120
Progress: 100.00% | Loss: 1.0052
Loss: 98.3365
Accuracy : 0.9867
Precision: 0.8331
Recall   : 0.8228

Epoch 75/120
Progress: 100.00% | Loss: 0.3671
Loss: 100.2541
Accuracy : 0.9867
Precision: 0.8352
Recall   : 0.8183

Epoch 76/120
Progress: 100.00% | Loss: 0.3177
Loss: 98.2542
Accuracy : 0.9868
Precision: 0.8390
Recall   : 0.8265

Epoch 77/120
Progress: 100.00% | Loss: 0.2259
Loss: 96.8201
Accuracy : 0.9869
Precision: 0.8407
Recall   : 0.8223
Model Saved!

Epoch 78/120
Progress: 100.00% | Loss: 1.0000
Loss: 96.9162
Accuracy : 0.9870
Precision: 0.8412
Recall   : 0.8232

Epoch 79/120
Progress: 100.00% | Loss: 0.2586
```

```
Loss: 98.4361
Accuracy : 0.9867
Precision: 0.8381
Recall   : 0.8194

Epoch 80/120
Progress: 100.00% | Loss: 1.0000
Loss: 97.4663
Accuracy : 0.9871
Precision: 0.8410
Recall   : 0.8255

Epoch 81/120
Progress: 100.00% | Loss: 0.5575
Loss: 97.0022
Accuracy : 0.9869
Precision: 0.8394
Recall   : 0.8251

Epoch 82/120
Progress: 100.00% | Loss: 0.1116
Loss: 95.0867
Accuracy : 0.9868
Precision: 0.8422
Recall   : 0.8207
Model Saved!

Epoch 83/120
Progress: 100.00% | Loss: 0.3421
Loss: 93.9801
Accuracy : 0.9871
Precision: 0.8418
Recall   : 0.8288
Model Saved!

Epoch 84/120
Progress: 100.00% | Loss: 0.1917
Loss: 94.0619
Accuracy : 0.9873
Precision: 0.8429
Recall   : 0.8289

Epoch 85/120
Progress: 100.00% | Loss: 0.1648
Loss: 93.4156
Accuracy : 0.9874
Precision: 0.8452
Recall   : 0.8303
Model Saved!
```

```
Epoch 86/120
Progress: 100.00% | Loss: 1.1200
Loss: 93.0495
Accuracy : 0.9874
Precision: 0.8463
Recall   : 0.8296
Model Saved!

Epoch 87/120
Progress: 100.00% | Loss: 0.1912
Loss: 91.6961
Accuracy : 0.9876
Precision: 0.8451
Recall   : 0.8366
Model Saved!

Epoch 88/120
Progress: 100.00% | Loss: 0.1304
Loss: 94.1979
Accuracy : 0.9872
Precision: 0.8427
Recall   : 0.8241

Epoch 89/120
Progress: 100.00% | Loss: 0.1357
Loss: 92.2497
Accuracy : 0.9876
Precision: 0.8466
Recall   : 0.8322

Epoch 90/120
Progress: 100.00% | Loss: 1.0000
Loss: 94.8309
Accuracy : 0.9874
Precision: 0.8496
Recall   : 0.8256

Epoch 91/120
Progress: 100.00% | Loss: 0.8120
Loss: 92.8439
Accuracy : 0.9875
Precision: 0.8463
Recall   : 0.8305

Epoch 92/120
Progress: 100.00% | Loss: 0.1112
Loss: 90.8339
Accuracy : 0.9879
Precision: 0.8506
Recall   : 0.8371
```

```
Model Saved!

Epoch 93/120
Progress: 100.00% | Loss: 1.0000
Loss: 89.0085
Accuracy : 0.9880
Precision: 0.8490
Recall   : 0.8407
Model Saved!

Epoch 94/120
Progress: 100.00% | Loss: 0.1658
Loss: 89.9204
Accuracy : 0.9878
Precision: 0.8477
Recall   : 0.8351

Epoch 95/120
Progress: 100.00% | Loss: 0.3043
Loss: 88.0159
Accuracy : 0.9880
Precision: 0.8558
Recall   : 0.8332
Model Saved!

Epoch 96/120
Progress: 100.00% | Loss: 0.1037
Loss: 87.4446
Accuracy : 0.9879
Precision: 0.8500
Recall   : 0.8413
Model Saved!

Epoch 97/120
Progress: 100.00% | Loss: 1.0053
Loss: 87.5473
Accuracy : 0.9882
Precision: 0.8536
Recall   : 0.8408

Epoch 98/120
Progress: 100.00% | Loss: 0.1879
Loss: 89.5666
Accuracy : 0.9880
Precision: 0.8491
Recall   : 0.8404

Epoch 99/120
Progress: 100.00% | Loss: 0.2640
Loss: 88.0471
```

```
Accuracy : 0.9881
Precision: 0.8522
Recall   : 0.8396

Epoch 100/120
Progress: 100.00% | Loss: 0.1690
Loss: 85.8294
Accuracy : 0.9881
Precision: 0.8519
Recall   : 0.8427
Model Saved!

Epoch 101/120
Progress: 100.00% | Loss: 1.0000
Loss: 89.6488
Accuracy : 0.9879
Precision: 0.8502
Recall   : 0.8398

Epoch 102/120
Progress: 100.00% | Loss: 1.0777
Loss: 87.1870
Accuracy : 0.9881
Precision: 0.8537
Recall   : 0.8425

Epoch 103/120
Progress: 100.00% | Loss: 0.2630
Loss: 88.8974
Accuracy : 0.9882
Precision: 0.8535
Recall   : 0.8368

Epoch 104/120
Progress: 100.00% | Loss: 0.4374
Loss: 86.1633
Accuracy : 0.9882
Precision: 0.8539
Recall   : 0.8440

Epoch 105/120
Progress: 100.00% | Loss: 0.1904
Loss: 84.9402
Accuracy : 0.9885
Precision: 0.8588
Recall   : 0.8448
Model Saved!

Epoch 106/120
Progress: 100.00% | Loss: 0.2139
```

```
Loss: 85.4794
Accuracy : 0.9885
Precision: 0.8553
Recall   : 0.8475

Epoch 107/120
Progress: 100.00% | Loss: 0.1378
Loss: 86.3675
Accuracy : 0.9882
Precision: 0.8523
Recall   : 0.8436

Epoch 108/120
Progress: 100.00% | Loss: 0.2372
Loss: 84.8211
Accuracy : 0.9884
Precision: 0.8594
Recall   : 0.8441
Model Saved!

Epoch 109/120
Progress: 100.00% | Loss: 0.2212
Loss: 83.3608
Accuracy : 0.9886
Precision: 0.8589
Recall   : 0.8493
Model Saved!

Epoch 110/120
Progress: 100.00% | Loss: 0.2532
Loss: 85.1984
Accuracy : 0.9885
Precision: 0.8565
Recall   : 0.8495

Epoch 111/120
Progress: 100.00% | Loss: 0.2087
Loss: 83.7566
Accuracy : 0.9886
Precision: 0.8575
Recall   : 0.8463

Epoch 112/120
Progress: 100.00% | Loss: 0.1420
Loss: 85.8515
Accuracy : 0.9885
Precision: 0.8579
Recall   : 0.8455

Epoch 113/120
```

```
Progress: 100.00% | Loss: 0.1416
Loss: 82.0869
Accuracy : 0.9890
Precision: 0.8611
Recall   : 0.8555
Model Saved!

Epoch 114/120
Progress: 100.00% | Loss: 0.1762
Loss: 85.0327
Accuracy : 0.9885
Precision: 0.8578
Recall   : 0.8451

Epoch 115/120
Progress: 100.00% | Loss: 1.0000
Loss: 87.1214
Accuracy : 0.9884
Precision: 0.8563
Recall   : 0.8475

Epoch 116/120
Progress: 100.00% | Loss: 0.2803
Loss: 81.9986
Accuracy : 0.9889
Precision: 0.8615
Recall   : 0.8531
Model Saved!

Epoch 117/120
Progress: 100.00% | Loss: 0.2899
Loss: 81.2543
Accuracy : 0.9888
Precision: 0.8592
Recall   : 0.8534
Model Saved!

Epoch 118/120
Progress: 100.00% | Loss: 1.0000
Loss: 80.6992
Accuracy : 0.9889
Precision: 0.8647
Recall   : 0.8519
Model Saved!

Epoch 119/120
Progress: 100.00% | Loss: 0.1867
Loss: 80.5027
Accuracy : 0.9891
Precision: 0.8638
```

```
Recall   : 0.8571
Model Saved!

Epoch 120/120
Progress: 100.00% | Loss: 0.3216
Loss: 83.2822
Accuracy : 0.9889
Precision: 0.8653
Recall   : 0.8481
```

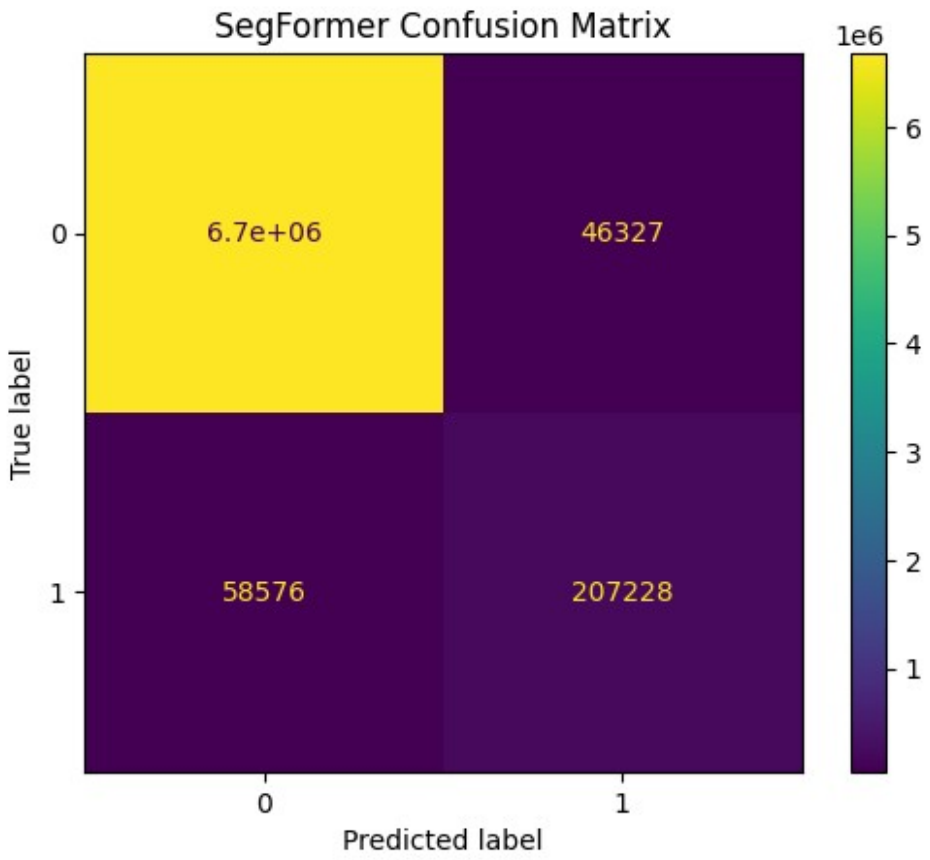
```
def calculate_all_metrics(pred, target):

    pred = (pred > 0.5).float()

    TP = (pred * target).sum()

    TN = ((1-pred) * (1-target)).sum()

    FP = (pred * (1-target)).sum()

    FN = ((1-pred) * target).sum()

    accuracy = (

        TP + TN

    ) / (

        TP + TN + FP + FN + 1e-8

    )

    precision = TP / (

        TP + FP + 1e-8

    )

    recall = TP / (

        TP + FN + 1e-8

    )

    f1 = (

        2 * TP

    ) / (

        2 * TP + FP + FN + 1e-8
```

```
    )
    dice = (
        2 * TP
    ) / (
        2 * TP + FP + FN + 1e-8
    )
    iou = TP / (
        TP + FP + FN + 1e-8
    )
    return (
        accuracy.item(),
        precision.item(),
        recall.item(),
        f1.item(),
        dice.item(),
        iou.item()
    )
model.load_state_dict(
    torch.load(
        "segformer_best.pth"
    )
)
model.eval()
acc = 0
prec = 0
rec = 0
f1_total = 0
```

```
dice_total = 0
iou_total = 0
count = 0
all_preds = []
all_targets = []
for images, masks in test_loader:
    images = images.to(device)
    masks = masks.to(device)
    with torch.no_grad():
        pred = model(images)
        pred = torch.sigmoid(pred)
        a, p, r, f1, d, i = calculate_all_metrics(
            pred,
            masks
        )
        acc += a
        prec += p
        rec += r
        f1_total += f1
        dice_total += d
        iou_total += i
        count += 1
        pred_bin = (
            pred > 0.5
        ).cpu().numpy().flatten()
        target_bin = masks.cpu().numpy().flatten()
```

```
        all_preds.extend(pred_bin)
        all_targets.extend(target_bin)
print("\nFINAL RESULTS\n")
print("Accuracy :", acc/count)
print("Precision:", prec/count)
print("Recall   :", rec/count)
print("F1 Score :", f1_total/count)
print("Dice     :", dice_total/count)
print("IoU      :", iou_total/count)
cm = confusion_matrix(
    all_targets,
    all_preds
)
ConfusionMatrixDisplay(cm).plot()
plt.title(
    "SegFormer Confusion Matrix"
)
plt.show()
```

```
FINAL RESULTS

Accuracy : 0.9850395850159905
Precision: 0.8097909844734452
Recall   : 0.7567697177556428
F1 Score : 0.7685987722467292
Dice     : 0.7685987722467292
IoU      : 0.6356836362657222
```




```
# ================ VALIDATION RESULTS ================

val_accuracy = acc / count
val_precision = prec / count
val_recall = rec / count
val_f1 = f1_total / count
val_dice = dice_total / count
val_iou = iou_total / count

print("\nVALIDATION RESULTS\n")

print("Validation Accuracy :", val_accuracy)
print("Validation Precision:", val_precision)
print("Validation Recall   :", val_recall)
print("Validation F1 Score :", val_f1)
print("Validation Dice     :", val_dice)
print("Validation IoU      :", val_iou)
```

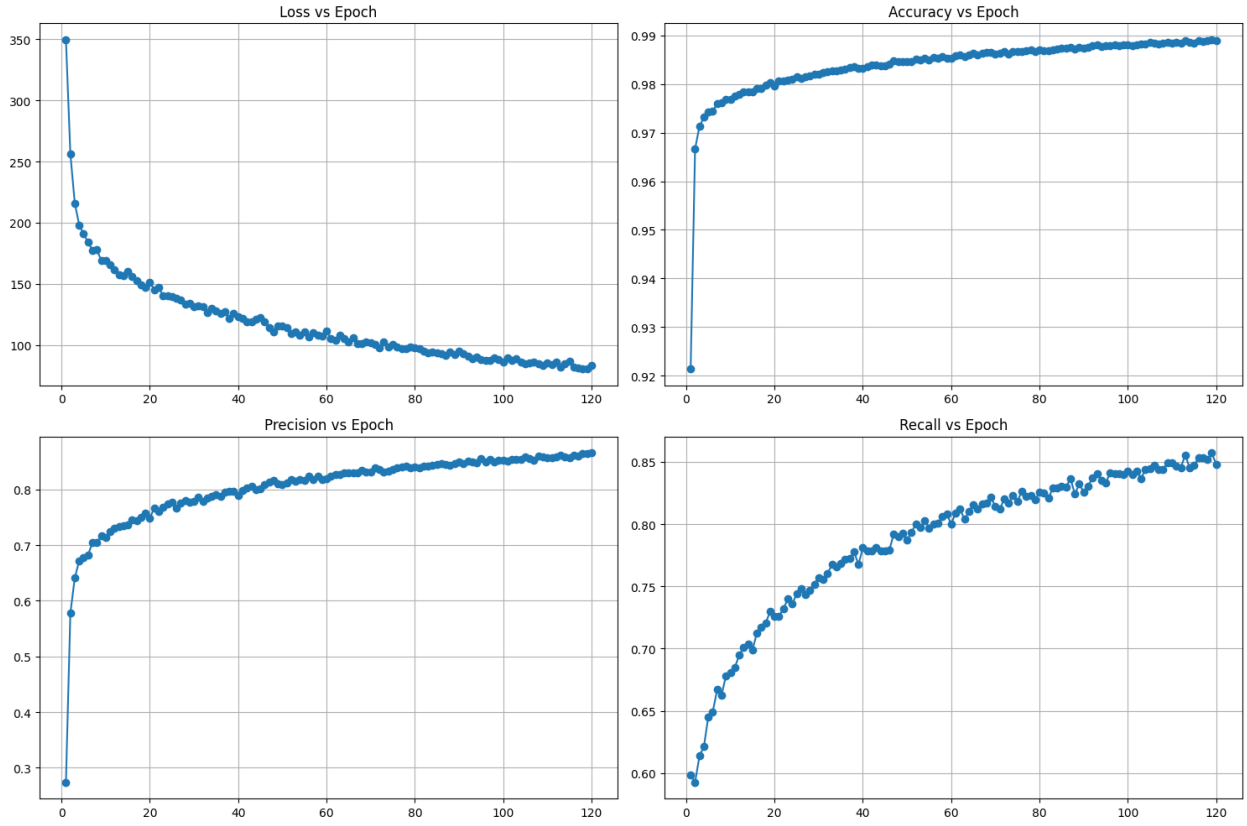
```
VALIDATION RESULTS

Validation Accuracy : 0.9850395850159905
Validation Precision: 0.8097909844734452
```

```
Validation Recall   : 0.7567697177556428
Validation F1 Score : 0.7685987722467292
Validation Dice     : 0.7685987722467292
Validation IoU      : 0.6356836362657222
```

```
epochs_range = range(
    1,
    len(train_losses)+1
)
plt.figure(figsize=(15,10))
# ================= LOSS =================
plt.subplot(2,2,1)
plt.plot(
    epochs_range,
    train_losses,
    marker='o'
)
plt.title(
    "Loss vs Epoch"
)
plt.grid()
# ================= ACCURACY =================
plt.subplot(2,2,2)
plt.plot(
    epochs_range,
    train_acc,
    marker='o'
)
plt.title(
```

```
    "Accuracy vs Epoch"
)
plt.grid()
# ================= PRECISION =================
plt.subplot(2,2,3)
plt.plot(
    epochs_range,
    train_prec,
    marker='o'
)
plt.title(
    "Precision vs Epoch"
)
plt.grid()
# ================= RECALL =================
plt.subplot(2,2,4)
plt.plot(
    epochs_range,
    train_rec,
    marker='o'
)
plt.title(
    "Recall vs Epoch"
)
plt.grid()
plt.tight_layout()
```

```
plt.show()
```



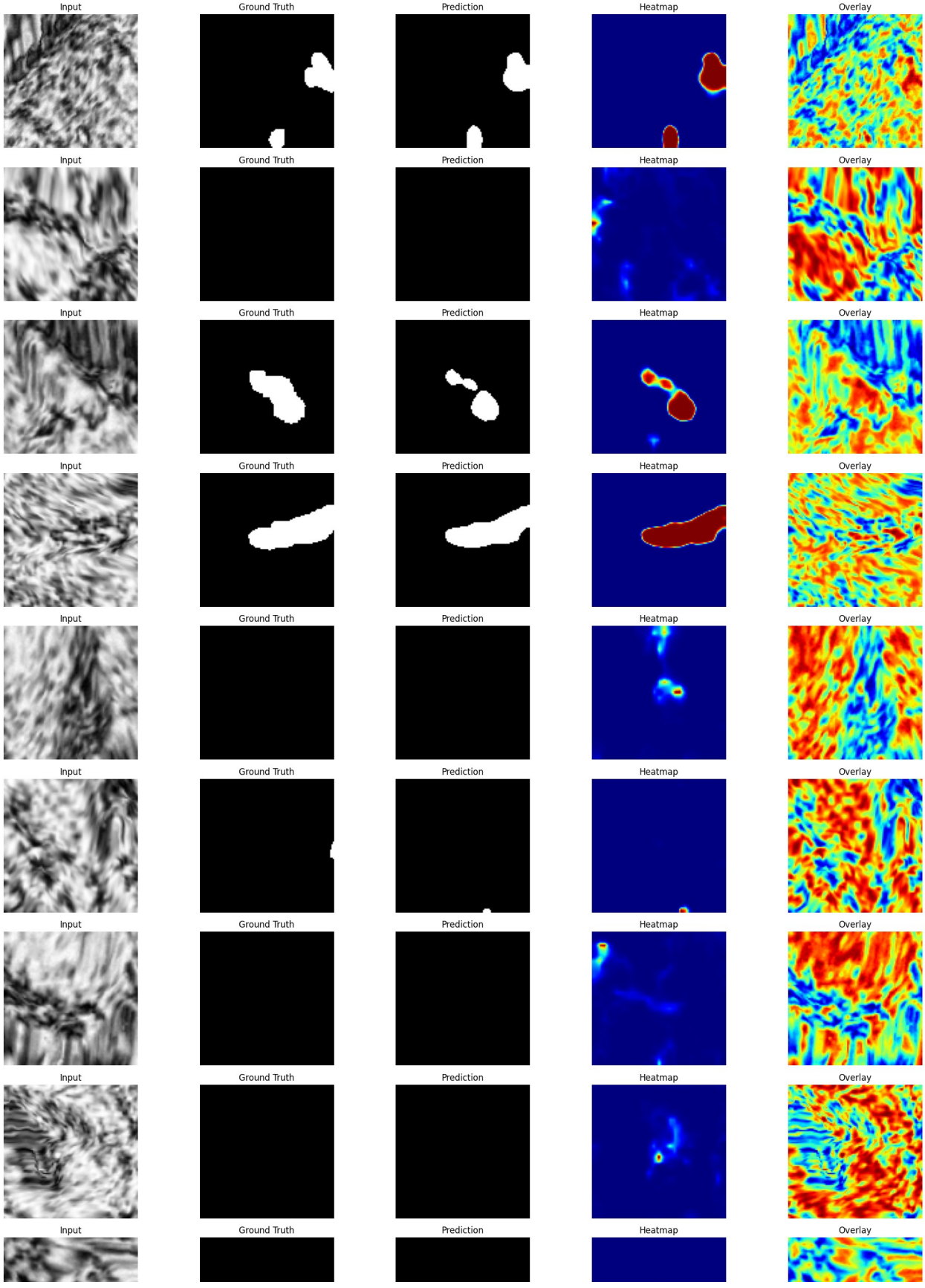

```
model.eval()

images_collected = 0

num_images = 10

all_inputs = []

all_masks = []

all_preds = []

for images, masks in test_loader:

    images = images.to(device)

    with torch.no_grad():

        pred = model(images)

        pred = torch.sigmoid(pred)
```

```
    all_inputs.extend(
        images.cpu().numpy()
    )
    all_masks.extend(
        masks.numpy()
    )
    all_preds.extend(
        pred.cpu().numpy()
    )
    images_collected += images.shape[0]
    if images_collected >= num_images:
        break
all_inputs = np.array(all_inputs)
all_masks = np.array(all_masks)
all_preds = np.array(all_preds)
plt.figure(figsize=(20,30))
for i in range(num_images):
    input_img = all_inputs[i][0]
    true_mask = all_masks[i][0]
    pred_mask = all_preds[i][0]
    binary_pred = (
        pred_mask > 0.5
    ).astype(float)
    heatmap = pred_mask
    overlay = (
        0.6 * input_img
```

```
        +
        0.4 * heatmap
    )
    # INPUT
    plt.subplot(
        num_images,
        5,
        i*5 + 1
    )
    plt.title("Input")
    plt.imshow(
        input_img,
        cmap='gray'
    )
    plt.axis('off')
    # GROUND TRUTH
    plt.subplot(
        num_images,
        5,
        i*5 + 2
    )
    plt.title("Ground Truth")
    plt.imshow(
        true_mask,
        cmap='gray'
    )
```

```
    plt.axis('off')
    # PREDICTION
    plt.subplot(
        num_images,
        5,
        i*5 + 3
    )
    plt.title("Prediction")
    plt.imshow(
        binary_pred,
        cmap='gray'
    )
    plt.axis('off')
    # HEATMAP
    plt.subplot(
        num_images,
        5,
        i*5 + 4
    )
    plt.title("Heatmap")
    plt.imshow(
        heatmap,
        cmap='jet'
    )
    plt.axis('off')
    # OVERLAY
```

```
    plt.subplot(
        num_images,
        5,
        i*5 + 5
    )
    plt.title("Overlay")
    plt.imshow(
        overlay,
        cmap='jet'
    )
    plt.axis('off')
plt.tight_layout()
plt.show()
```


Input
Ground Truth
Prediction
Heatmap
Overlay
Input
Ground Truth
Prediction
Heatmap
Overlay
Input
Ground Truth
Prediction
Heatmap
Overlay
Input
Ground Truth
Prediction
Heatmap
Overlay
Input
Ground Truth
Prediction
Heatmap
Overlay
Input
Ground Truth
Prediction
Heatmap
Overlay
Input
Ground Truth
Prediction
Heatmap
Overlay
Input
Ground Truth
Prediction
Heatmap
Overlay
Input
Ground Truth
Prediction
Heatmap
Overlay